

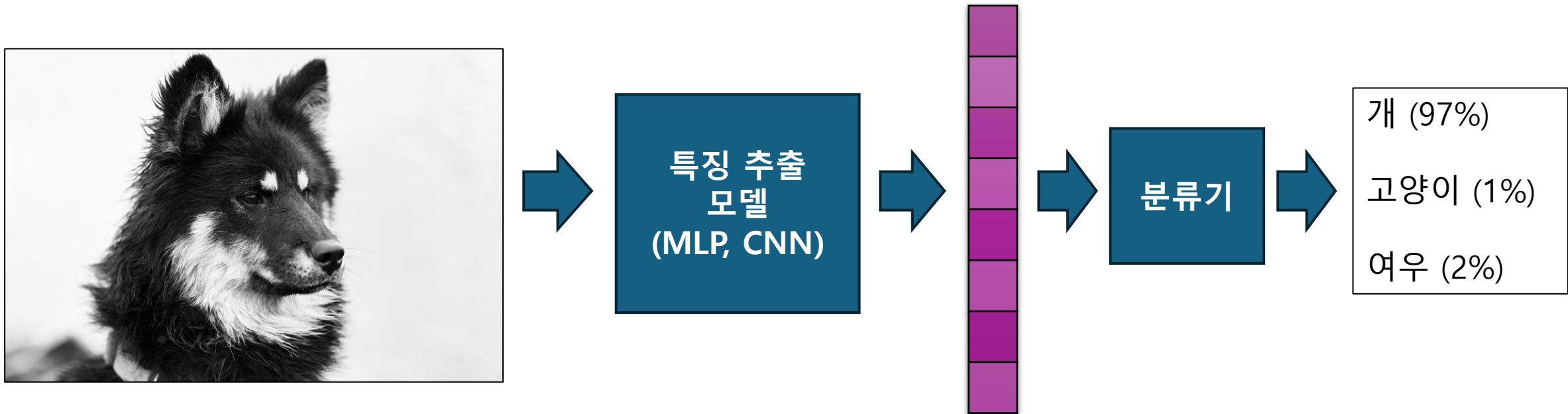
모두를 위한 딥러닝 (이미지처리편)

경북대학교 IT교육센터

허 찬

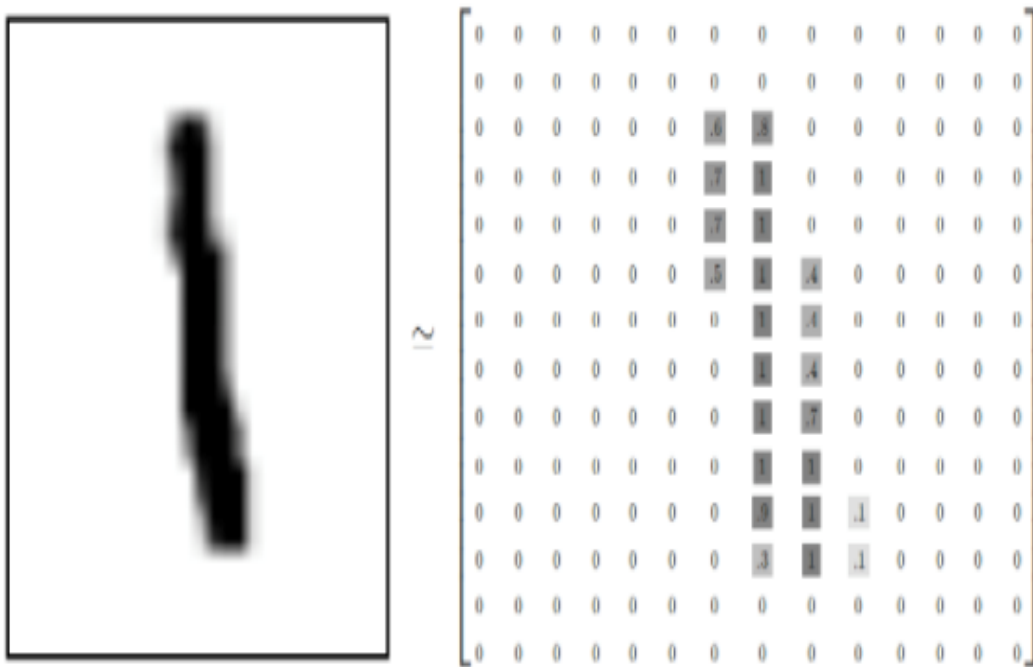
차시(모듈)	차시별 주제	학습내용
1	딥러닝을 이용한 이미지 처리	이미지 처리의 이해 다양한 이미지처리의 응용 분야에 대한 이해
2	특징 추출의 이해	이미지 데이터 전처리 기법의 이해 이미지 특징 추출의 이해 및 실습 딥러닝을 이용한 학습 및 추론 과정의 이해
3	Convolution Neural Network	합성곱 신경망의 이해
4	CNN 모델의 이해	다양한 CNN 모델의 이해 딥러닝의 여러가지 요소 이해
5	CNN을 이용한 분류 모델 실습	CNN을 이용한 이미지 분류 모델 실습
6	전이 학습 모델의 이해	전이 학습의 이해 전이학습을 이용한 이미지 처리 모델 실습
7	객체 인식 모델의 이해	다양한 객체 인식 모델의 이해 Semantic segmentation (U-Net)의 이해
8	Transformer 기반 이미지 처리 모델 1	Attention과 Transformer의 이해
9	Transformer 기반 이미지 처리 모델 2	Transformer 기반 이미지 처리 모델 (ViT)의 이해
10	이미지 생성 모델의 이해	최신 이미지 생성 모델의 이해 오토인코더의 이해 및 실습
11	최신 이미지 처리 기술의 이해	최신 컴퓨터비전 모델의 이해 비디오 처리의 이해

딥러닝을 이용한 이미지 처리



3.1 딥러닝을 이용한 손글씨 데이터 분류

데이터셋 설명



MNIST dataset

문제 정의:

- 0~9까지의 손글씨 데이터를 보고 어떤 숫자인지 맞추는 분류 모델 설계

데이터 정의

Input data:

- $[0,0,0,0,\dots,0.6,0.8,0,0,0,\dots,0.7,\dots,0,0,0.1,0,0,\dots]$ 로 이루어진 60000개 학습 데이터/10000개 테스트 데이터.

Label:

- $[1,0,0,0,0,0,0,0,0,0]$

3.1 딥러닝을 이용한 손글씨 데이터 분류

1. Module import

```
from keras.datasets import mnist
from keras.models import Sequential
from keras import layers
from keras.utils import to_categorical
import matplotlib.pyplot as plt
from keras.utils import plot_model
```

3.1 딥러닝을 이용한 손글씨 데이터 분류

2. Data preprocessing

MNIST 데이터셋 불러오기

```
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

이미지 데이터 준비하기 (모델에 맞는 크기로 바꾸고 0과 1사이로 스케일링)

```
train_images = train_images.reshape((60000, 28 * 28))
```

```
train_images = train_images.astype('float32') / 255
```

```
test_images = test_images.reshape((10000, 28 * 28))
```

```
test_images = test_images.astype('float32') / 255
```

```
print(train_labels.shape)
```

```
print(test_labels.shape)
```

레이블을 범주형으로 인코딩

```
train_labels = to_categorical(train_labels)
```

```
test_labels = to_categorical(test_labels)
```

mnist.load_data 함수:

- 손글씨 데이터셋을 불러오는 함수

train_images.reshape 함수:

- 이미지를 한 줄의 vector로 형태 변경

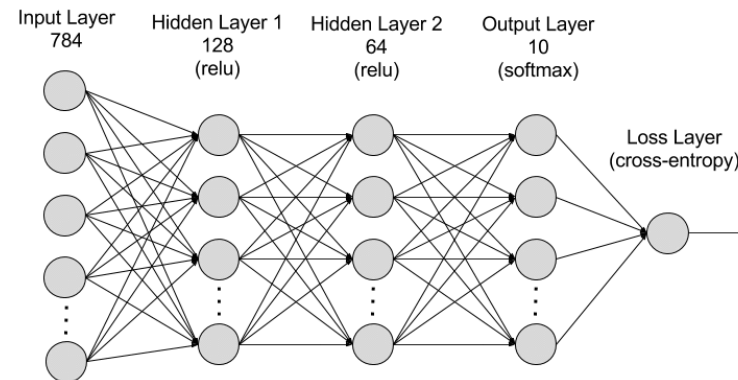
3.1 딥러닝을 이용한 손글씨 데이터 분류

3. Model build

모델 정의하기 (여기에서는 Sequential 클래스 사용)

```
model = Sequential()
model.add(layers.Dense(100, activation='relu', input_shape=(28 * 28,)))
model.add(layers.Dense(150, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.summary()
```

- model.add() : 신경망을 쌓는 함수



3.1 딥러닝을 이용한 손글씨 데이터 분류

4. Model compile & training

모델 컴파일 하기

```
model.compile(optimizer='rmsprop',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

fit() 함수로 모델 훈련 시키기

```
hist=model.fit(train_images, train_labels, epochs=3, batch_size=64, verbose=1)
```

- Epochs: 전체 데이터를 몇 번 반복하여 학습시킬지 정하는 파라미터
- batch_size: 한 번에 학습하는 데이터의 개수를 지정하는 파라미터
- model.fit은 return값을 주면 학습 과정의 loss 그래프를 그릴 수 있음 (옵션)

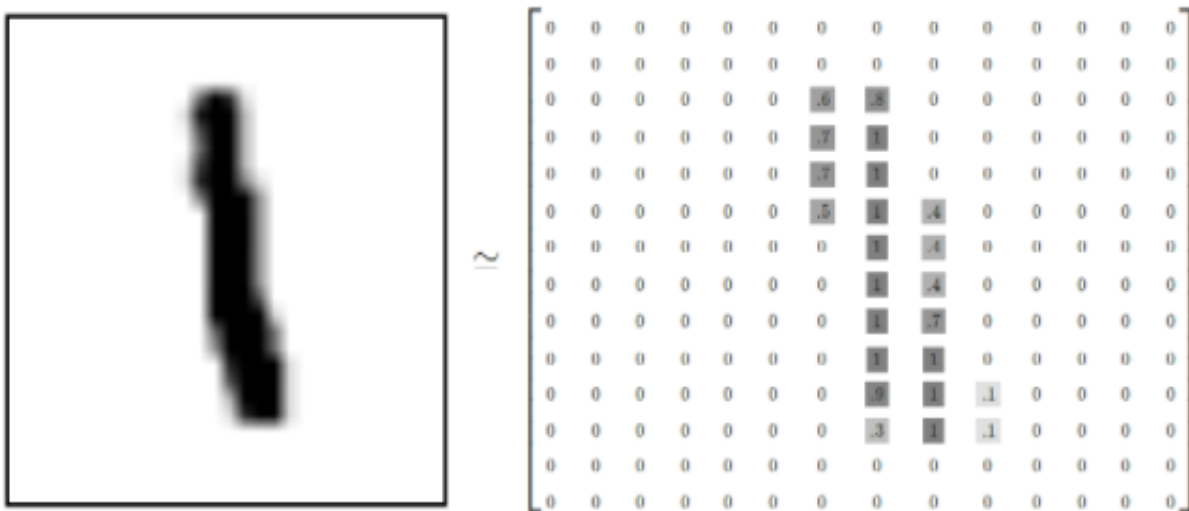
3.1 딥러닝을 이용한 손글씨 데이터 분류

5. Model test

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=0)
print('loss:', test_loss)
print('test_accuracy:', test_acc)
```

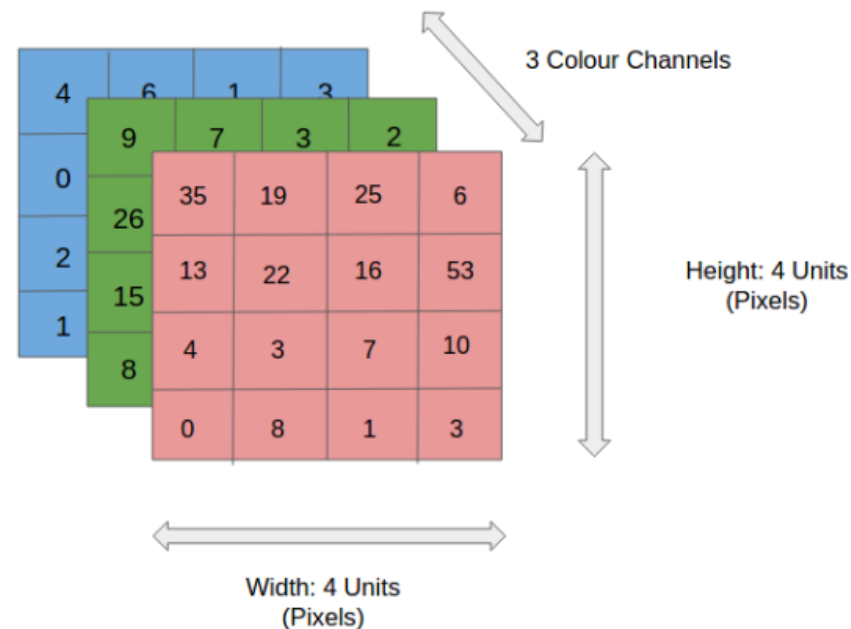
Convolution Neural Network (CNN)

3.2 CNN (Convolution Neural Network)이란?



Flatten 방식 Input data:

- 쪽 펼쳐진 input
[0,0,0,0,.....0.6,0.8,0,0,0,....0.7....0,0,0.1,0,0,..]



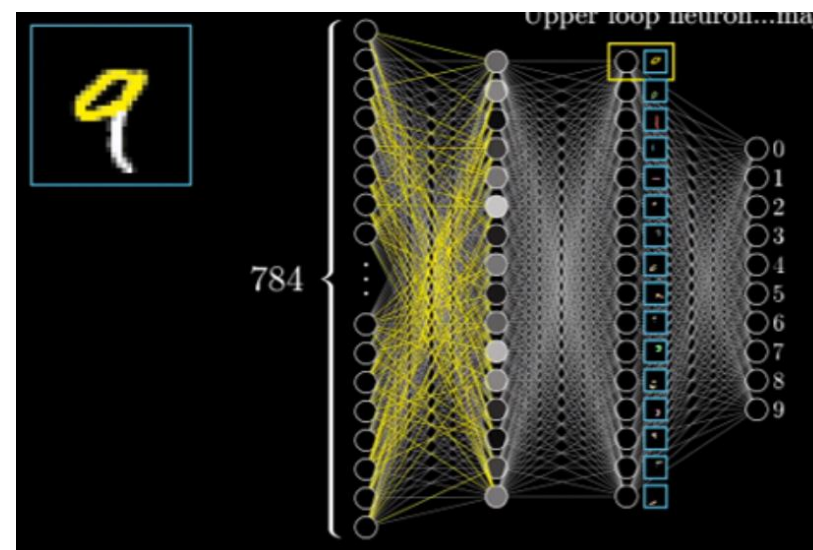
Matrix 방식 Input data:

- Matrix (보통 N*N, N=224)

3.2 CNN (Convolution Neural Network)이란?

단순한 DNN을 이용하였을 경우(MLP layer) 이미지 처리의 문제

- Input data가 크기때문에 연산량이 과도하게 요구
Ex) $28 \times 28 = 784$ 개의 특징
- Layer를 2~3개만 늘려도 연산량이 크게 증가
Ex) $784 \times 512 = 401408$ parameter
- 이미지 안의 위치정보를 알 수 없음



3.2 CNN (Convolution Neural Network)이란?

A bit of history:

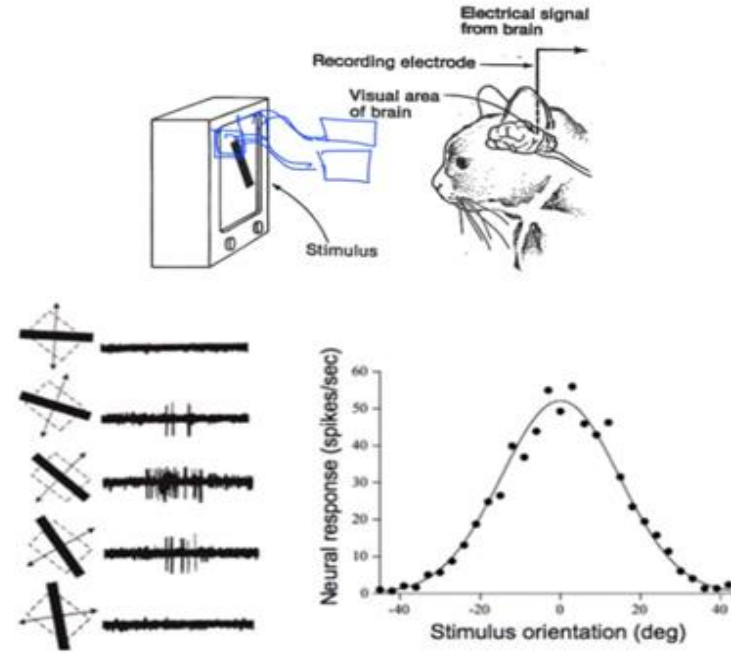
Hubel & Wiesel,
1959

RECEPTIVE FIELDS OF SINGLE
NEURONES IN
THE CAT'S STRIATE CORTEX

1962

RECEPTIVE FIELDS, BINOCULAR
INTERACTION
AND FUNCTIONAL ARCHITECTURE IN
THE CAT'S VISUAL CORTEX

1968...

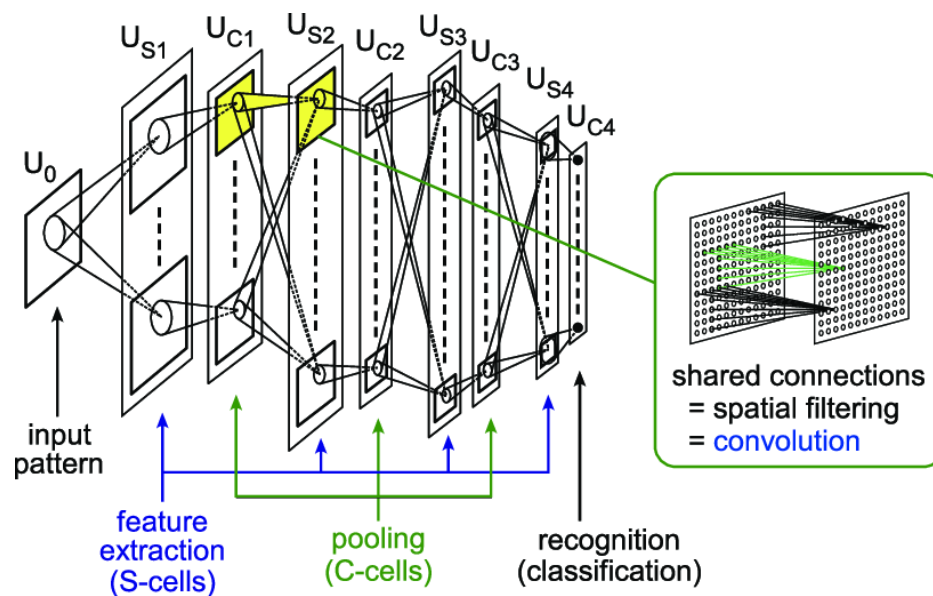


- 1959년, 허벨&위젤, 고양이가 어떻게 이미지를 인식하는지 알아보기 위해 실험을 진행
- 보통 사람이 이미지를 볼 때는 각 뉴런들이 각 이미지의 서로 다른 부분적인 조각을 담당
- 하나의 이미지를 인지한다는 건 하나의 이미지를 여러 시각세포를 통해 얻은 조각 정보를 종합하여 인식.

3.2 CNN (Convolution Neural Network)이란?

1980년대:

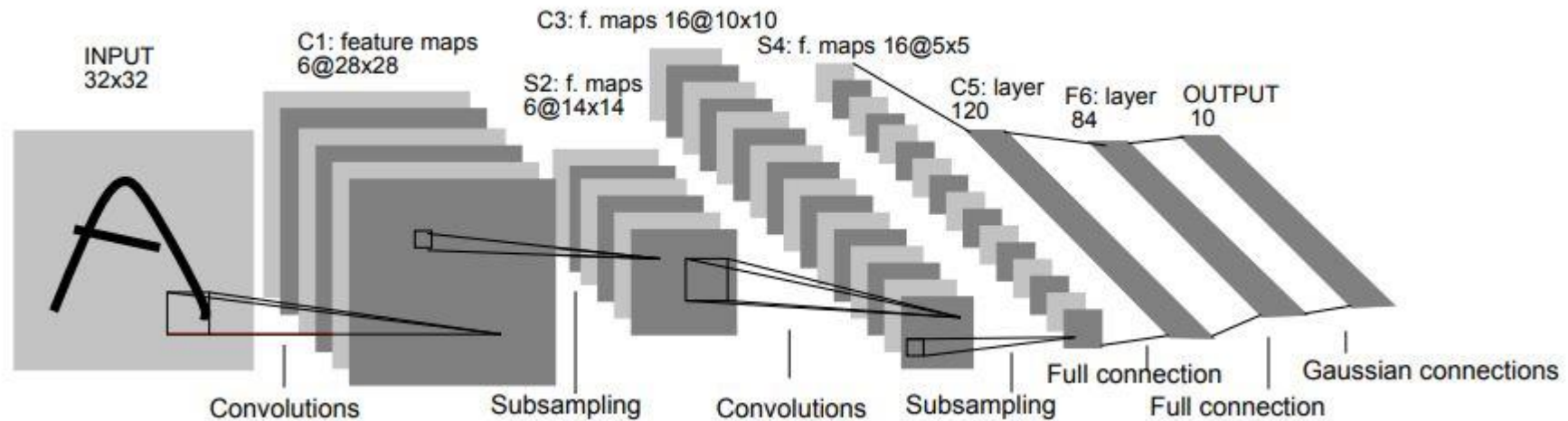
- 일본의 후쿠시마 쿠니히코가 '네오코그니트론'을 개발.
- 이는 패턴 인식을 위한 신경망 모델로, 후에 CNN의 설계에 영향을 끼침.
- 네오코그니트론은 계층적 구조를 가지며, 각 계층은 여러 수용 필드를 통해 정보를 처리.



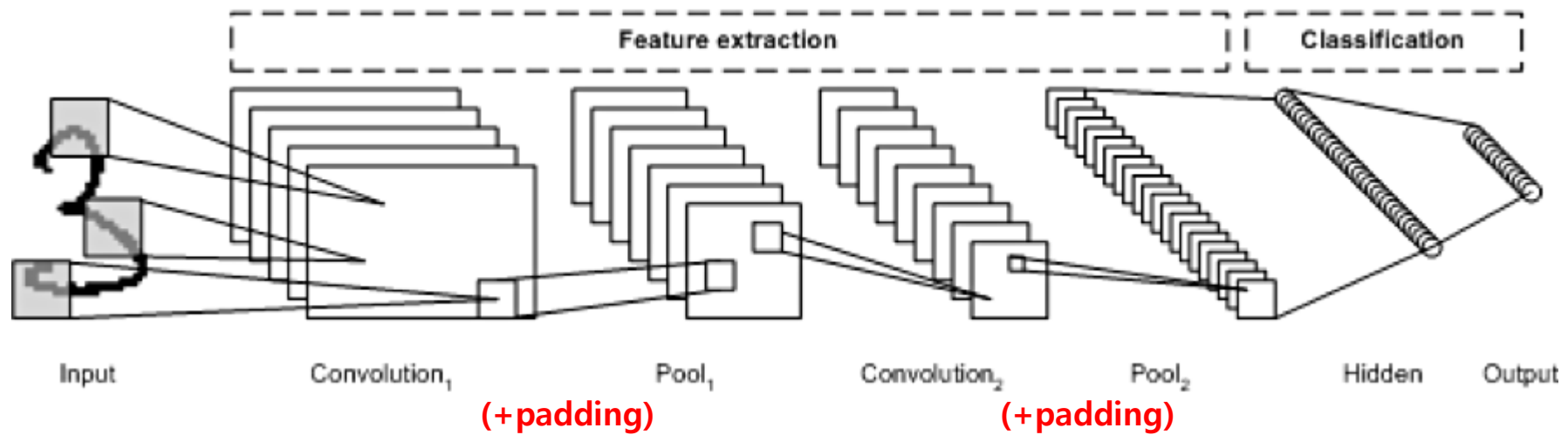
3.2 CNN (Convolution Neural Network)이란?

CNN의 탄생

- **1998년**: 프랑스의 연구자인 얀 르쿤(Yann LeCun)이 LeNet-5를 제안, 현대적인 CNN의 탄생.
- LeNet-5는 손글씨 숫자 인식을 목표로 개발되었으며, 실제 우편 번호 인식 시스템에서 상업적으로 사용.
- 이 모델은 합성곱 계층과 풀링 계층, 완전 연결 계층을 포함하는 현대적인 CNN의 기본 구조.

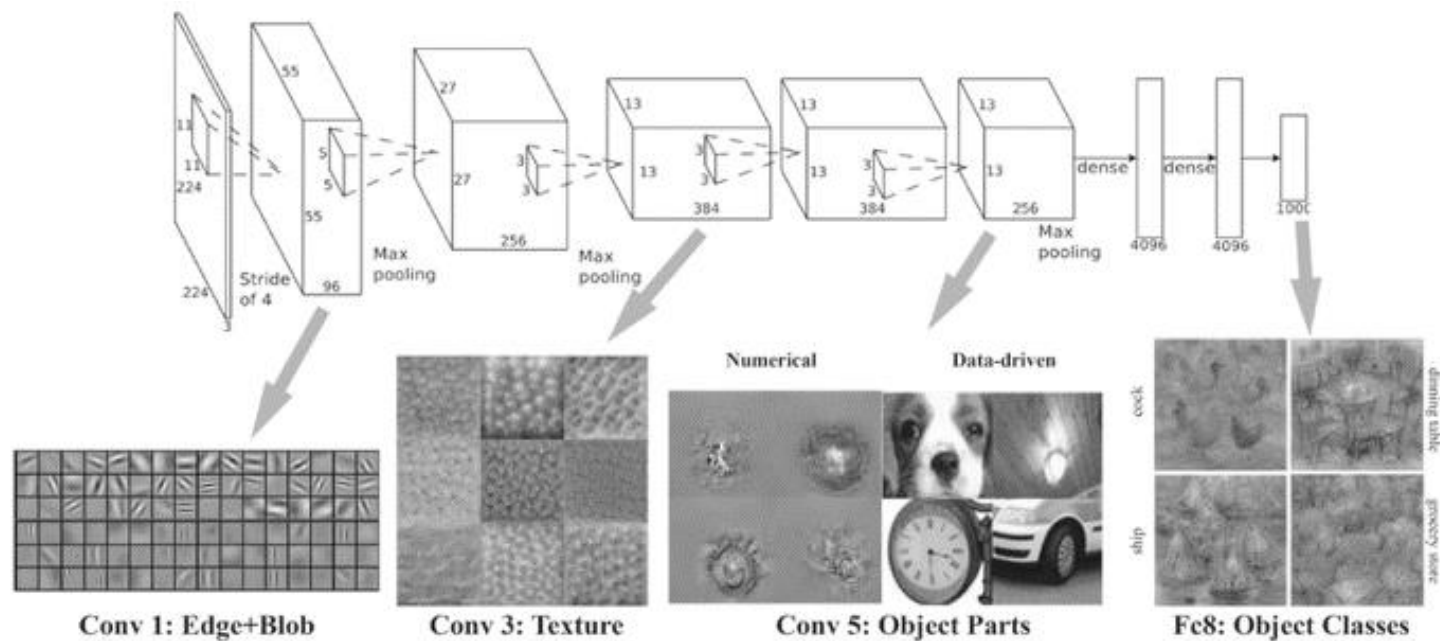


3.2 CNN (Convolution Neural Network)이란?



- CNN의 구조

3.2 CNN (Convolution Neural Network)이란?

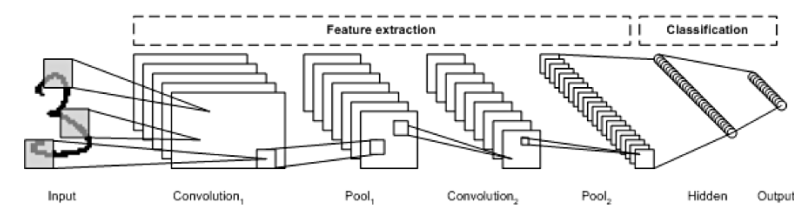
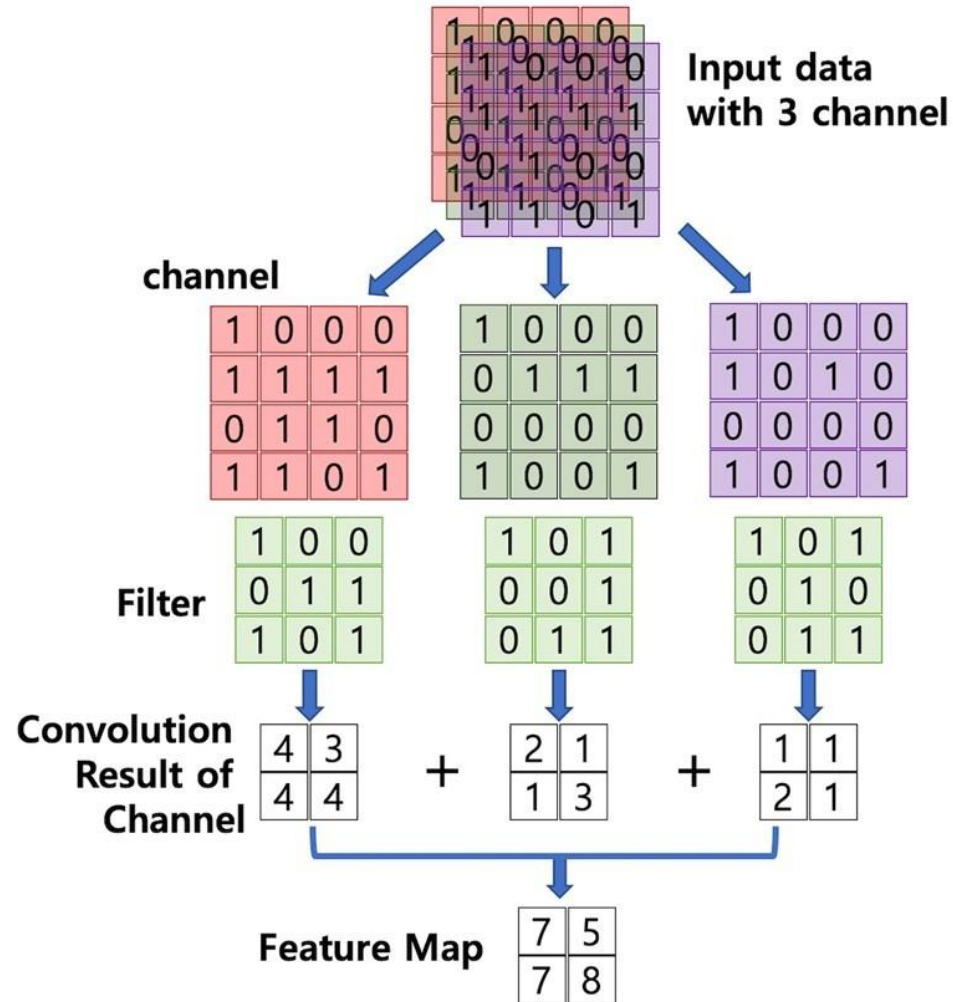


- 이때 CNN layer가 깊어질수록 Input으로 들어간 이미지는 더 많은 Convolutional Layer를 거치게 되면서, **Filter가 발견하는 패턴은 더욱 구체화됨.**
- 첫 Layer에서 Filter가 발견한 패턴이 Edge, Corner, Circle 등이었다면 이것이 점점 깊어질수록 **눈, 귀, 머리카락, 새들의 털, 부리** 등 훨씬 구체적인 패턴을 발견할 수 있음.

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 필터링과 컨볼루션

- 필터의 각 요소들이 학습!



3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 필터링과 컨볼루션

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

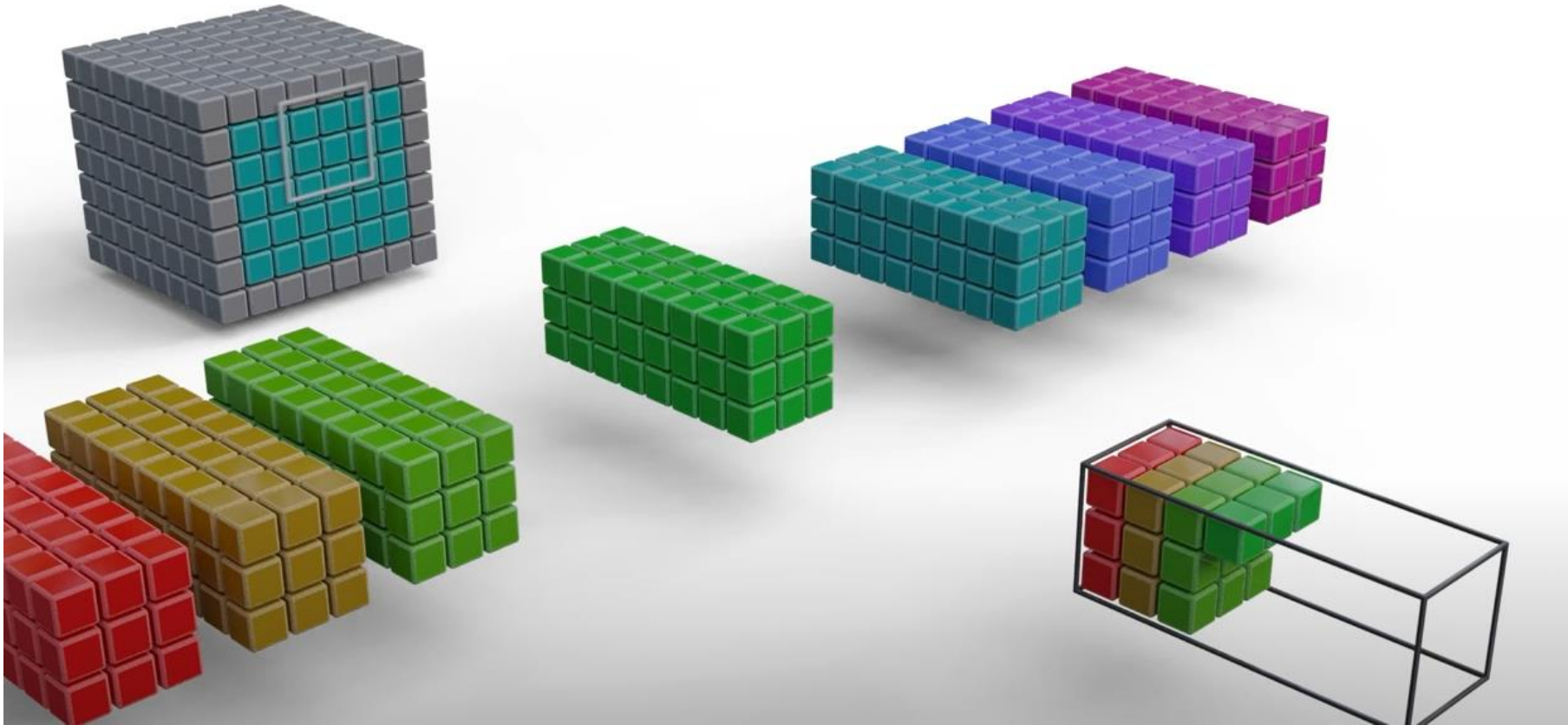
4		

Convolved
Feature

- Stride 1 : 이동거리 1

3.2 CNN (Convolution Neural Network)이란?

컨볼루션 시각화 영상

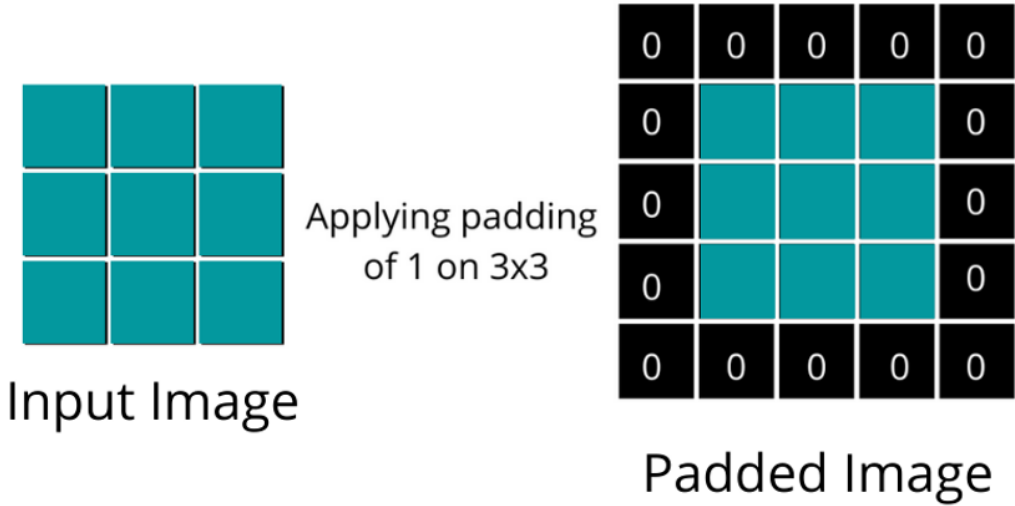
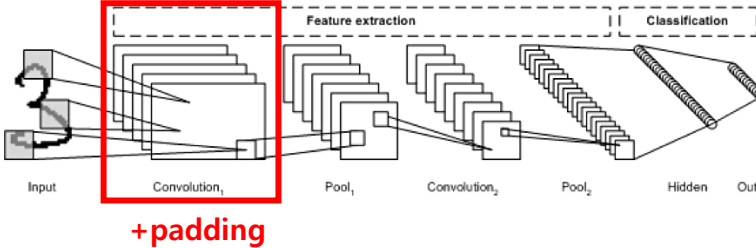


<https://www.youtube.com/watch?v=w4kNHKcBGzA&t=192s>

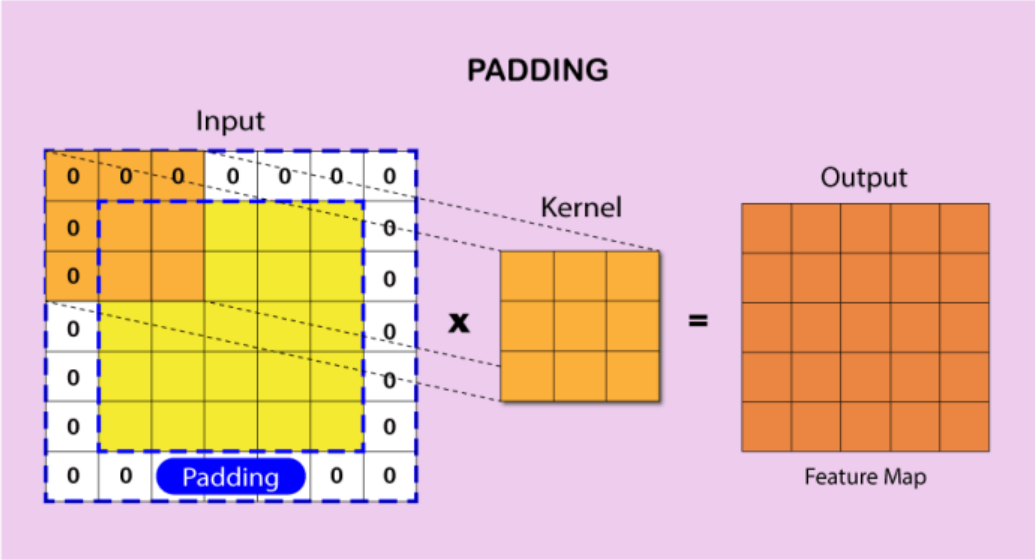
- 여러 필터를 사용해서 특징 맵을 추출

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소(optional): 패딩 (padding)



Padding 과정



Padding 과정을 거친 output

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 패딩 (padding)

1. 크기 유지

- **크기 감소 방지:** 합성곱 연산을 수행할 때, 필터의 크기와 스트라이드 설정에 따라 출력의 차원이 입력의 차원보다 작아지게 됨.
- 패딩을 추가함으로써 입력 데이터의 크기를 인위적으로 늘려, 출력 데이터의 크기가 입력 데이터의 크기와 동일하게 유지되도록 할 수 있음.
- 이는 네트워크의 깊이가 깊어질 때 크기가 너무 작아지는 것을 방지하며, 이를 통해 많은 합성곱 계층을 쌓을 수 있음.

2. 정보 손실 최소화

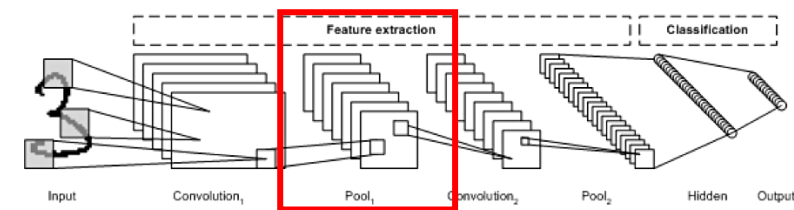
- **가장자리 정보 보존:** 합성곱 연산은 입력 데이터의 지역적 특성을 추출하는 과정, 패딩이 없는 경우 데이터의 가장자리 부분은 중앙 부분에 비해 필터와 덜 겹치게 되므로 정보 손실 가능.
- 패딩을 추가하면 이러한 가장자리 부분의 정보도 충분히 활용할 수 있어, 이미지의 가장자리에 있는 중요한 특징도 잘 추출될 수 있음.

3. 디자인 유연성

- **다양한 필터 크기와 스트라이드 조절:** 패딩을 조절함으로써 다양한 크기의 필터와 스트라이드를 효율적으로 사용할 수 있습니다. 이는 네트워크 설계 시 보다 유연한 구조를 가능하게 하며, 특정 응용에 맞춰 최적의 네트워크 구성을 탐색할 수 있습니다.

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 풀링 (pooling)



13	20	30	0
8	12	3	0
34	70	33	5
111	80	10	23

Activation Map

20	30
111	33

Max Pooling

13	8
66	18

Average Pooling

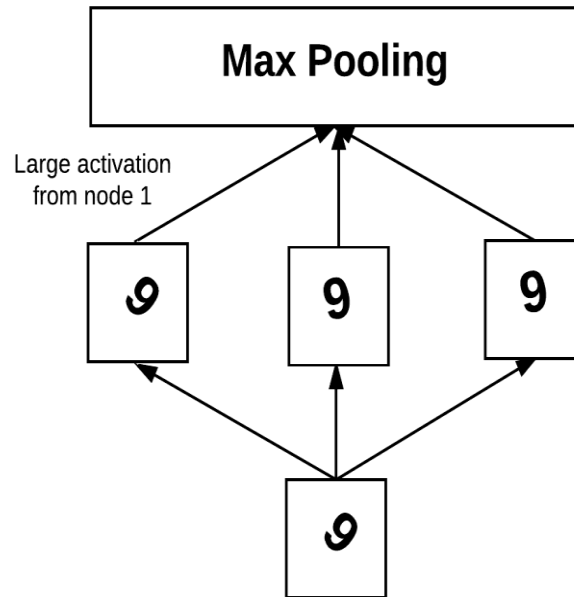
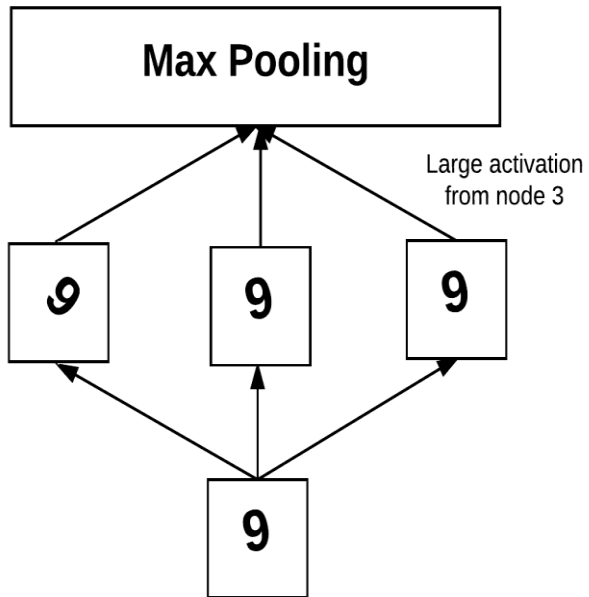
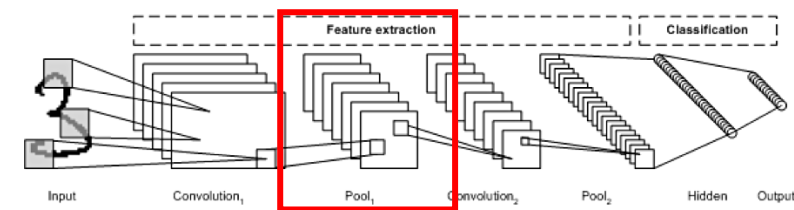
8	0
34	5

Min Pooling

- 여러가지 pooling 방법

3.2 CNN (Convolution Neural Network)이란?

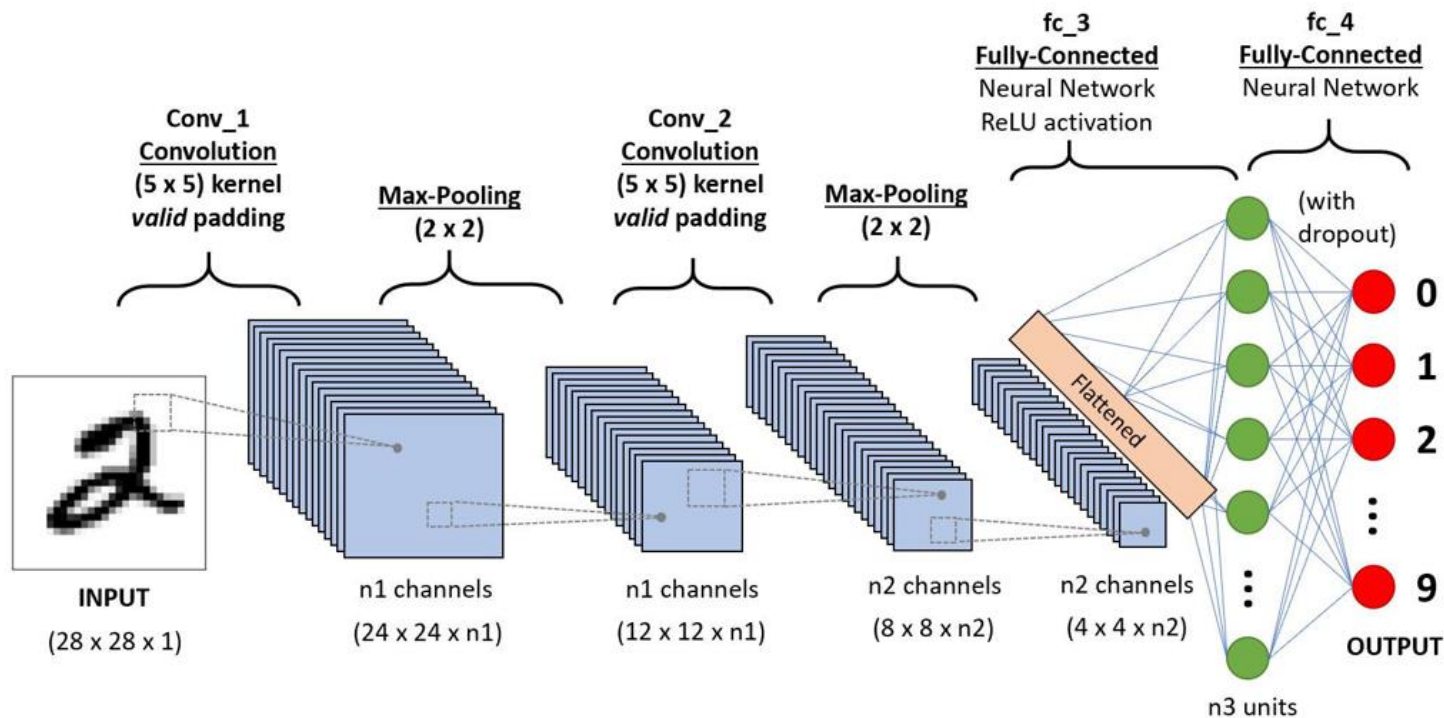
CNN의 요소: 풀링 (pooling)



- 풀링은 모델이 input data의 민감한 변화도 잘 적응하게 만든다 (강건하다)

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 풀링 (pooling)



- Pooling을 해도 feature map의 개수 (차원)은 줄어들지 않음!
- Pooling은 학습되는 과정이 아님!

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 풀링 (pooling)

- CNN에서 풀링(pooling) 과정은 중요한 역할을 수행.
- 풀링은 합성곱 층에서 추출된 특징 맵(feature map)을 다운샘플링하여 차원을 축소시키는 과정.
- 주로 사용되는 풀링 방법에는 최대 풀링(max pooling)과 평균 풀링(average pooling)이 있음.
- 풀링 과정은 몇 가지 중요한 이유로 인해 널리 사용:

1. 불변성 증가

- **변환 불변성(Translation Invariance):** 풀링은 입력 이미지에서 위치가 약간 변화한 특징들에 대해 민감하지 않도록 함.
- 예를 들어, 객체가 이미지 내에서 약간 이동하거나 회전했을 때도 같은 특징을 감지할 수 있도록 도와줍니다. 이는 모델이 일반화를 잘 하도록 만들어, 실제 환경에서의 성능을 개선.

2. 계산 효율성 증가

- **차원 축소:** 풀링을 통해 특징 맵의 크기를 줄이면, 그 이후의 네트워크 층에서 처리해야 할 데이터의 양이 감소. 이는 필요한 메모리 용량과 계산 비용을 줄이며, 모델의 학습과 실행 시간을 단축.

3.2 CNN (Convolution Neural Network)이란?

CNN의 요소: 풀링 (pooling)

3. 과적합 방지

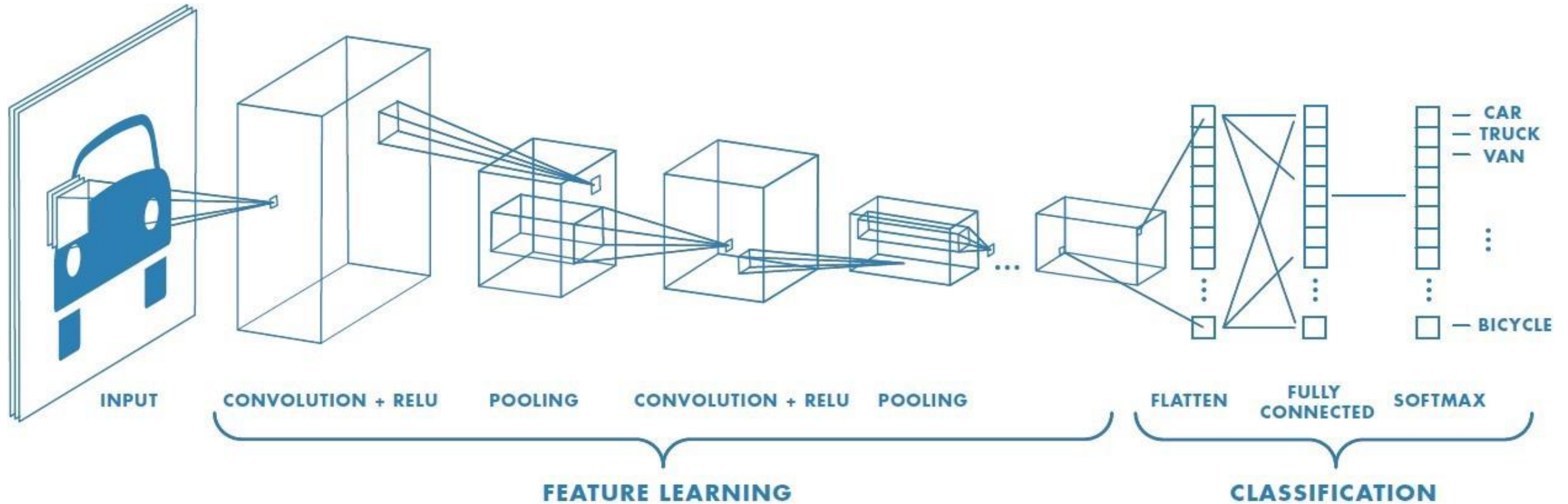
- **특징의 강화와 추상화:** 풀링 과정은 모델이 특정 특징들에 대해 너무 자세히 학습하는 것을 방지하고, 더 추상적인 수준에서 특징을 인식하도록 돕는 기능.
- 이는 모델이 학습 데이터에 과적합되는 것을 방지하고, 새로운 데이터에 대한 일반화 능력을 향상.

4. 강인성 증가

- **노이즈 및 변동에 대한 강인성:** 풀링 과정은 노이즈와 작은 변동에 대해 모델을 더 강인하게 만드는 역할.
- 예를 들면, 최대 풀링은 지역적으로 가장 강한 신호만을 선택하여 다음 계층으로 전달하기 때문에, 노이즈가 포함된 특징은 자연스럽게 제거.

3.2 CNN의 기본 아키텍처

기본적인 CNN 구조



- Convolution+Activation (+padding) -> Pooling -> Convolution+Activation (+padding) -> Pooling -> Flatten -> Fully Connected -> Softmax

그림: <https://prod.velog.io>

3.3 CNN을 이용한 손글씨 데이터 분류 모델

데이터셋 설명

- 목적: 손글씨로 쓰인 숫자를 CNN 모델이 인식하여 테스트 데이터에 대해 어떤 숫자인지 예측
- 6만개의 학습 데이터셋
- 1만개의 테스트 데이터셋

Input:

- 28x28 픽셀로 이루어진 이미지 데이터

Output:

- 0~9 중 1개의 숫자



MNIST dataset

3.3 CNN을 이용한 손글씨 데이터 분류 모델

1) 패키지 import 및 dataset load

```
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D
from sklearn.preprocessing import LabelEncoder
from keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

print(X_train.shape)
print(y_train.shape)
```

3.3 CNN을 이용한 손글씨 데이터 분류 모델

2) data 전처리 및 label one-hot encoding

```
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1).astype('float32') / 255
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1).astype('float32') / 255

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

print(X_train.shape)
print(y_train.shape)
```

3.3 CNN을 이용한 손글씨 데이터 분류 모델

3) Model build

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(3,3), input_shape = (28,28,1), activation='relu'))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Dropout(0.25))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(y_train.shape[1], activation='softmax')) ## 분류문제의 마지막 activation은 항상 softmax ##
model.summary()

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=3, batch_size=24, verbose=1) ## 작업중 기록을 보기위한 학습 정보 저장
```


3.3 CNN을 이용한 손글씨 데이터 분류 모델

4) 모델 평가 및 history 확인

```
print("model accuracy : %.4f" % model.evaluate(X_test, y_test)[1])

y_vloss = history.history['val_loss']
y_loss = history.history['loss']

x_len = np.arange(len(y_loss))

plt.plot(x_len, y_vloss, marker='.', c='red', label='val_set_loss')
plt.plot(x_len, y_loss, marker='.', c='blue', label='train_set_loss')
plt.legend()
plt.xlabel('epochs')
plt.ylabel('loss')
plt.grid()
plt.show()
```

- 학습 진행시 Loss 추세 확인 (과적합 확인용)

감사합니다